

```
import os
import torch
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
dataset_path = "/content/drive/MyDrive/ANNlab/Pistachio_Image_Dataset/Pistachio_Image_Dataset"
```

```
# Define image transformations
transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.125])
])
```

```
class PistachioDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.images = []
        self.labels = []

        #Load images from "Kirmizi_Pistachio" folder (Class 0)
        kirmizi_folder = os.path.join(root_dir, "Kirmizi_Pistachio")
        for file in os.listdir(kirmizi_folder):
            if file.lower().endswith(('jpg', 'png', 'jpeg')):
                self.images.append(os.path.join(kirmizi_folder, file))
                self.labels.append(0) # 0 for Kirmizi_Pistachio

        # Load images from "Siirt_Pistachio" folder (Class 1)
        siirt_folder = os.path.join(root_dir, "Siirt_Pistachio")
        for file in os.listdir(siirt_folder):
            if file.lower().endswith(('jpg', 'png', 'jpeg')):
                self.images.append(os.path.join(siirt_folder, file))
                self.labels.append(1) # 1 for Siirt_Pistachio

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        # Load image and label
        img_path = self.images[idx]
        label = self.labels[idx]

        # Open image
        image = Image.open(img_path).convert("L")

        # Apply transformations
        if self.transform:
            image = self.transform(image)

        # Convert label to tensor
        label = torch.tensor(label, dtype=torch.long)

        return image, label
```

```
# Instantiate dataset
dataset = PistachioDataset(dataset_path, transform=transform)
```

```
# Create DataLoader
dataloader = DataLoader(dataset, batch_size=64, shuffle=True)
```

```
# Test the DataLoader
for images, labels in dataloader:
    print(f"Batch size: {images.shape}, Labels: {labels}")
    break
```

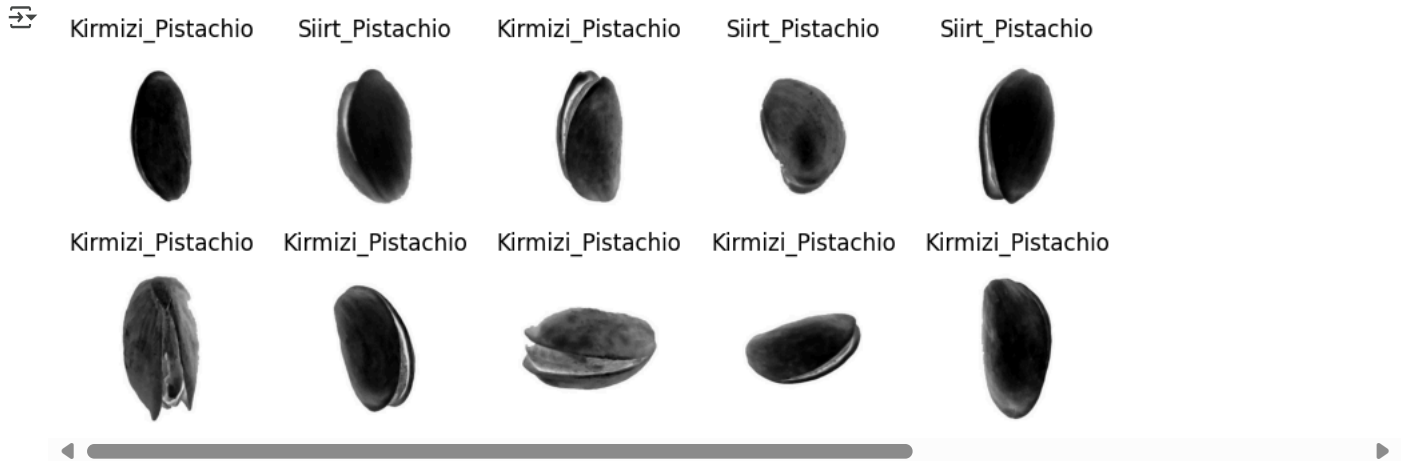
Batch size: torch.Size([64, 1, 256, 256]), Labels: tensor([1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0])

```

image, labels = next(iter(dataloader))
class_name = ['Kirmizi_Pistachio', 'Siirt_Pistachio']
plt.figure(figsize = (10,10))
for i in range(10):
    plt.subplot(5,5,i+1)
    img = image[i].permute(1,2,0)
    img = img *0.123 + 0.5
    plt.imshow(img.numpy(), cmap = 'Grays')
    plt.title(f"{class_name[labels[i].item()]}")
    plt.axis("off")

plt.show()

```



```

X = image.numpy()
y = labels.numpy()

X = X.reshape(X.shape[0], -1)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

import tensorflow as tf
from tensorflow.keras import layers, models, regularizers

# Build model
model = models.Sequential([
    layers.Input(shape=(X_train.shape[1],)),
    layers.Dense(256, activation='relu', kernel_regularizer=regularizers.l2(0.001)),
    layers.Dropout(0.3),
    layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.001)),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid') # Binary classification
])

# Compile
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Train
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=50, batch_size=64, verbose=1)

```



```

1/1 ----- 1s 612ms/step - accuracy: 0.8431 - loss: 11.0744 - val_accuracy: 0.8462 - val_loss: 10.1638
Epoch 33/50
1/1 ----- 1s 638ms/step - accuracy: 0.8039 - loss: 7.2896 - val_accuracy: 0.7692 - val_loss: 19.9625
Epoch 34/50
1/1 ----- 1s 551ms/step - accuracy: 0.7647 - loss: 22.1039 - val_accuracy: 0.8462 - val_loss: 14.6979
Epoch 35/50
1/1 ----- 1s 627ms/step - accuracy: 0.8039 - loss: 18.1732 - val_accuracy: 0.8462 - val_loss: 7.0919
Epoch 36/50
1/1 ----- 1s 639ms/step - accuracy: 0.7843 - loss: 13.7061 - val_accuracy: 1.0000 - val_loss: 0.9494
Epoch 37/50
1/1 ----- 1s 551ms/step - accuracy: 0.8627 - loss: 8.8862 - val_accuracy: 0.9231 - val_loss: 2.9012
Epoch 38/50
1/1 ----- 1s 558ms/step - accuracy: 0.8039 - loss: 17.3399 - val_accuracy: 0.9231 - val_loss: 3.1261
Epoch 39/50
1/1 ----- 1s 553ms/step - accuracy: 0.8039 - loss: 20.7423 - val_accuracy: 1.0000 - val_loss: 0.9540
Epoch 40/50
1/1 ----- 1s 564ms/step - accuracy: 0.8627 - loss: 15.7444 - val_accuracy: 0.8462 - val_loss: 4.0606
Epoch 41/50
1/1 ----- 1s 545ms/step - accuracy: 0.8627 - loss: 5.7538 - val_accuracy: 0.8462 - val_loss: 10.1931
Epoch 42/50
1/1 ----- 1s 643ms/step - accuracy: 0.9216 - loss: 2.7746 - val_accuracy: 0.8462 - val_loss: 12.0052
Epoch 43/50
1/1 ----- 1s 539ms/step - accuracy: 0.8039 - loss: 8.3516 - val_accuracy: 0.8462 - val_loss: 10.4630
Epoch 44/50
1/1 ----- 1s 650ms/step - accuracy: 0.8824 - loss: 9.7061 - val_accuracy: 0.8462 - val_loss: 7.7911
Epoch 45/50
1/1 ----- 1s 1s/step - accuracy: 0.8039 - loss: 9.3167 - val_accuracy: 0.8462 - val_loss: 4.8283
Epoch 46/50
1/1 ----- 1s 885ms/step - accuracy: 0.8824 - loss: 4.3022 - val_accuracy: 0.9231 - val_loss: 2.0827
Epoch 47/50
1/1 ----- 1s 1s/step - accuracy: 0.9020 - loss: 4.4597 - val_accuracy: 0.9231 - val_loss: 1.6978
Epoch 48/50
1/1 ----- 1s 540ms/step - accuracy: 0.8039 - loss: 11.7554 - val_accuracy: 0.8462 - val_loss: 2.9809
Epoch 49/50
1/1 ----- 1s 570ms/step - accuracy: 0.8824 - loss: 5.3863 - val_accuracy: 0.8462 - val_loss: 5.6092
Epoch 50/50
1/1 ----- 1s 611ms/step - accuracy: 0.9020 - loss: 5.8679 - val_accuracy: 0.8462 - val_loss: 7.2144
<keras.src.callbacks.history.History at 0x789d65e34550>

```

```

# Predict labels on test set
pred_probs = model.predict(X_test)
pred_labels = (pred_probs > 0.5).astype(int)

```

```

# Display some predictions
class_names = ['Kirmizi_Pistachio', 'Siirt_Pistachio']
for i in range(10):
    print(f"Actual: {class_names[y_test[i]]}, Predicted: {class_names[int(pred_labels[i])]}")

```

```

↔ 1/1 ----- 0s 100ms/step
Actual: Kirmizi_Pistachio, Predicted: Kirmizi_Pistachio
Actual: Kirmizi_Pistachio, Predicted: Kirmizi_Pistachio
Actual: Kirmizi_Pistachio, Predicted: Kirmizi_Pistachio
Actual: Kirmizi_Pistachio, Predicted: Kirmizi_Pistachio
Actual: Kirmizi_Pistachio, Predicted: Siirt_Pistachio
Actual: Siirt_Pistachio, Predicted: Siirt_Pistachio
Actual: Kirmizi_Pistachio, Predicted: Kirmizi_Pistachio
Actual: Kirmizi_Pistachio, Predicted: Siirt_Pistachio
Actual: Kirmizi_Pistachio, Predicted: Kirmizi_Pistachio
Actual: Siirt_Pistachio, Predicted: Siirt_Pistachio
<ipython-input-18-328395193994>:8: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will
    print(f"Actual: {class_names[y_test[i]]}, Predicted: {class_names[int(pred_labels[i])]}")

```

Start coding or [generate](#) with AI.

+ Code

+ Text

Start coding or [generate](#) with AI.

